🔍 Diff Commits

(Comparando versões do seu projeto)

Aqui você deixa de ver só “o que mudou agora” e passa a ver:

o que mudou ao longo do tempo

🧠 Ideia central

`git diff` entre commits = comparar duas versões do projeto

👉 É como perguntar:

- “o que mudou desde ontem?”
- “qual alteração quebrou o sistema?”
- “o que foi feito nessa versão?”

🧬 1. Comparar dois commits

📌 Comando:

```
git diff <commit1> <commit2>
```

🔧 Exemplo

```
git diff a1b2c3 d4e5f6
```

🧠 O que ele mostra?

👉 Diferença completa entre dois pontos da história

📄 Saída exemplo:

```
- preço = 10
+ preço = 20
```

🔍 2. `git show` — ver um commit específico

📌 Comando:

```
git show <commit>
```

🔧 Exemplo

```
git show HEAD
```

🧠 O que ele mostra?

- mudanças feitas
- mensagem do commit
- autor
- data

💡 Uso real

👉 “O que exatamente foi feito nesse commit?”

📃 3. Comparar com o commit anterior

📌 Comando:

```
git show <commit>
```

👉 por padrão ele compara com o commit anterior

🔧 Exemplo

```
git show HEAD~1
```

🧠 Tradução

- HEAD → commit atual
- HEAD~1 → 1 commit atrás
- HEAD~2 → 2 commits atrás

💧 4. Comparar um arquivo específico

📌 Comando:

```
git diff <commit> arquivo.txt
```

🔧 Exemplo

```
git diff HEAD~1 app.txt
```

🧠 Uso

👉 ver evolução de um único arquivo

📊 5. Ver resumo das mudanças

📌 Comando:

```
git diff <commit> --stat
```

ou:

```
git show <commit> --stat
```

🔧 Exemplo

```
git show HEAD --stat
```

📄 Saída:

```
app.txt | 2 +-
```

🧠 Tradução

- + → linhas adicionadas
- - → linhas removidas

🧠 6. Como identificar commits

Seu PDF mostra várias formas importantes:

◇ Usando HEAD

```
git diff HEAD~1 HEAD
```

◇ Usando branch

```
git diff main
```

◇ Usando ID

```
git diff 3e887ab
```

◇ Usando remoto

```
git diff origin/main
```

🔥 Exemplo completo (vida real)

```
echo "v1" > app.txt
git add .
git commit -m "versão 1"

echo "v2" >> app.txt
git add .
git commit -m "versão 2"

git diff HEAD~1 HEAD
```

📄 Resultado:

```
+ v2
```

🧠 Cenário real (nível profissional)

Você tem um bug:

👉 “parou de funcionar depois de alguma mudança”

🔍 Investigação:

```
git log --oneline
```

Escolhe commits:

```
git diff commit_antigo commit_novo
```

🎯 Resultado:

👉 Você descobre exatamente:

- o que mudou
- onde mudou
- quem mudou

💡 Uso avançado (muito importante)

🔎 Encontrar erro

Compare:

```
git diff versão_boa versão_ruim
```

🔎 Revisão de código

Antes de aceitar mudanças:

```
git diff branch1 branch2
```

🔎 Antes de deploy

```
git diff main origin/main
```

⚠ Erros comuns

❌ não comparar versões

→ difícil achar bugs

❌ não usar HEAD corretamente

→ confusão total

🎯 Regra de ouro

Sempre compare antes de tomar decisão

🧠 Nível engenharia

Quem domina isso:

- debuga rápido
- entende evolução do sistema
- trabalha melhor em equipe

🚀 Agora você já domina

Você aprendeu:

- diff local
- diff staged
- diff entre commits

- análise de histórico

👉 Isso já é nível profissional